

Lab 03: Advanced Power Query

Lab 03: Advanced Power Query and Data Transformation

Lab summary

These labs teach reusable Power Query transformation patterns using synthetic monthly order files.

Azure Government readiness

The required labs are **Gov-ready** because they use local files and Power BI Desktop. Query folding against external sources, dataflows, Dataflows Gen2, incremental refresh in the Service, and source connectors are **Verify for Gov** or **Commercial-focused** depending on the feature.

Power BI project format

Build Power BI artifacts as PBIP projects. PBIP is the source-controlled format for this workshop. PBIX files can be generated from PBIP later if a packaged file is needed.

Lab data

File or folder	Description
https://raw.githubusercontent.com/Coding-Forge/PBI-Advanced-Factory/main/data/monthly-orders/orders-2026-01.csv	January order extract.
https://raw.githubusercontent.com/Coding-Forge/PBI-Advanced-Factory/main/data/monthly-orders/orders-2026-02.csv	February order extract.
https://raw.githubusercontent.com/Coding-Forge/PBI-Advanced-Factory/main/data/monthly-orders/orders-2026-03.csv	March order extract with a data quality issue.
https://raw.githubusercontent.com/Coding-Forge/PBI-Advanced-Factory/main/data/reference/product-category-map.csv	Product category reference table.

Use **Get data > Web** in Power BI Desktop to load each CSV from the raw GitHub URL.

Novice-friendly how-to guide

Create a Power Query parameter

1. In Power BI Desktop, select **Home > Transform data**.
2. In Power Query, select **Manage Parameters > New Parameter**.
3. Enter the exact parameter name from the lab.
4. Choose the data type, such as Text or Date/Time.
5. Enter the current value.
6. Select **OK**.
7. Use the parameter in source or filter steps instead of hard-coded values.

Module parameter reference

Create these five parameters during Lab 03. The required Web-source path uses `RawDataBaseUrl` , `EnvironmentName` , `RangeStart` , and `RangeEnd` . `SourceFolderPath` is included as an optional placeholder for offline or folder-based delivery; it can be blank and does not need to be used in the Web-source path.

Parameter	Type	Suggested value	Used in core Web path?	Purpose
<code>RawDataBaseUrl</code>	Text	<code>https://raw.githubusercontent.com/Coding-Forge/PBI-Advanced-Factory/main/data/</code>	Yes	Base path for raw GitHub CSV files.
<code>SourceFolderPath</code>	Text	Blank, or a local folder path for offline delivery	No	Optional folder connector placeholder for instructor/offline scenarios.
<code>EnvironmentName</code>	Text or List	<code>Dev</code>	Yes, as documentation/concept	Introduces Dev/Test/Prod source switching without changing the required Web-source path.
<code>RangeStart</code>	Date/Time	<code>2026-01-01 00:00:00</code>	Yes, for incremental refresh prep	Lower bound for incremental refresh filtering.
<code>RangeEnd</code>	Date/Time	<code>2026-04-01 00:00:00</code>	Yes, for incremental refresh prep	Upper bound for incremental refresh filtering.

Append queries

1. In Power Query, select **Home > Append Queries > Append Queries as New**.
2. Choose **Three or more tables** when combining several monthly files.
3. Add each raw monthly query to the append list.
4. Select **OK**.
5. Rename the appended query using the lab's exact query name.
6. Check that column names and data types match.

Disable load for intermediate queries

1. Right-click each raw or staging query.
2. Clear **Enable load**.
3. Leave only final model-ready queries enabled.
4. Select **Close & Apply** after reviewing query names and data types.

Exercise 1: Staged query architecture

Objective: Create maintainable query layers.

Tasks

1. Create a parameter named `RawDataBaseUrl` with value `https://raw.githubusercontent.com/Coding-Forge/PBI-Advanced-Factory/main/data/`.
2. Create Web queries for the three monthly order files using the raw URLs.
3. Name the raw Web queries `raw_Orders_2026_01`, `raw_Orders_2026_02`, and `raw_Orders_2026_03`.
4. Append the three raw queries into a staged query named `stg_OrdersCombined`.
5. Create a final load query named `FactOrders`.
6. Disable load for raw and staging queries.

Expected result

Only the final model-ready query is loaded. Intermediate queries remain available for development and troubleshooting.

Concept note: M language fundamentals

Every Power Query transformation is an M step that references the previous step by name. The generated UI usually creates tables, but M can also work with lists and records, such as the list passed to `Table.Combine` or the record of options passed to `Csv.Document`. When a generated step becomes hard to read, rename the step and review the formula bar before hand-editing in Advanced Editor.

Exercise 2: Folder combine

Objective: Combine monthly files safely.

Tasks

1. Load each monthly order CSV using the Web connector.
2. Add a source file name column to each raw query for lineage.
3. Append the monthly queries.
4. Confirm all appended queries share the same schema.
5. Apply explicit data types.

Expected result

All valid monthly order rows are appended into one query with consistent types and source lineage.

Exercise 3: Parameters and source switching

Objective: Use parameters to avoid hard-coded local paths.

Tasks

1. Create `SourceFolderPath` as a Text parameter. Leave it blank unless you are using an offline/local folder delivery path.
2. Create `EnvironmentName` as a Text or List parameter with values such as `Dev` , `Test` , and `Prod` .
3. Keep the required Web-source queries pointed at `RawDataBaseUrl` .
4. Document how each environment would map to source paths, gateway connections, or alternate base URLs.

Expected result

The required Web-source query remains stable, and the model documents how it could be repointed for offline folder delivery or environment-specific sources.

Exercise 4: Custom functions

Objective: Encapsulate reusable cleanup logic.

Tasks

1. Create a query named `fn_CleanText` .
2. Convert it to a function that accepts nullable text.
3. Trim whitespace.
4. Clean non-printable characters.
5. Return null safely when input is null.
6. Invoke the function against text columns such as `CustomerName` , `SalesChannel` , `ProductName` , `ProductCategory` , and `ProductSubcategory` .

Example function

```
(inputText as nullable text) as nullable text =>
let
    Result =
        if inputText = null then
            null
        else
            Text.Proper(Text.Trim(Text.Clean(inputText)))
in
    Result
```

Invocation examples

Use **Transform > Invoke Custom Function** in Power Query, or add a `Table.TransformColumns` step in Advanced Editor.

Example for the final order fact query:

```
CleanedOrderText =
    Table.TransformColumns(
        TypedOrders,
        {
            {"CustomerName", each fn_CleanText(_), type nullable text},
            {"SalesChannel", each fn_CleanText(_), type nullable text}
        }
    )
```

Example for the product reference query:

```
CleanedProductText =
    Table.TransformColumns(
        dim_ProductCategory,
        {
            {"ProductName", each fn_CleanText(_), type nullable text},
            {"ProductCategory", each fn_CleanText(_), type nullable text},
            {"ProductSubcategory", each fn_CleanText(_), type nullable text}
        }
    )
```

Validation note: In the current `pbi-local` PBIP model, `fn_CleanText` is not yet present or invoked. This Lab 03 lab step is where learners create and apply it.

Expected result

Text cleanup is reusable, null-safe, and applied consistently to selected customer, channel, and product text fields.

Exercise 5: Data quality and error handling

Objective: Detect and handle invalid source rows.

Tasks

1. Add explicit type conversions.
2. Identify rows with invalid quantity, price, or order date values.
3. Create an error review query named `err_OrdersReview`.
4. Keep `err_OrdersReview` load disabled unless the instructor wants to show it in the model for review.
5. Create `FactOrders` from rows that do not have data quality issues.
6. Add a data quality notes section to the lab output.

`err_OrdersReview` Power Query pattern

Create `err_OrdersReview` as a **Reference** of `stg_OrdersCombined`, not as a duplicate. The query should add a readable reason column and keep only rows with one or more issues.

```
let
    Source = stg_OrdersCombined,
    AddedDataQualityIssue =
        Table.AddColumn(
            Source,
            "DataQualityIssue",
            each
                let
                    ParsedOrderDate = try Date.From([OrderDate]) otherwise null,
                    ParsedQuantity = try Number.From([Quantity]) otherwise null,
                    ParsedUnitPrice = try Number.From([UnitPrice]) otherwise null,
                    ProductCodeText = try Text.Trim(Text.From([ProductCode])) otherwise "",
                    Issues =
                        List.RemoveNulls(
                            {
                                if ParsedOrderDate = null then "Missing or invalid OrderDate" else null,
                                if ParsedQuantity = null or ParsedQuantity <= 0 then "Missing or non-positive Quantity" else null,
                                if ParsedUnitPrice = null or ParsedUnitPrice <= 0 then "Missing or non-positive UnitPrice" else null,
                                if ProductCodeText = "" then "Missing ProductCode" else null
                            }
                        )
                in
                    Text.Combine(Issues, "; ")
            , type text
        ),
    ErrorRows =
        Table.SelectRows(
            AddedDataQualityIssue,
            each [DataQualityIssue] <> ""
        )
in
    ErrorRows
```

Use the same validation logic as the starting point for `FactOrders`, but keep only rows where `DataQualityIssue` is blank before applying final data types:

```
ValidRows =  
    Table.SelectRows(  
        AddedDataQualityIssue,  
        each [DataQualityIssue] = ""  
    )
```

Expected result

Learners can distinguish between silently removing bad data and explicitly reviewing data quality issues.

Concept note: business-rule checks

Technical type errors are only one kind of data quality issue. Add readable checks for business rules such as non-positive quantities, missing product codes, or invalid dates, and keep `err_OrdersReview` available so reviewers can see which rows were excluded and why.

Exercise 6: Query folding

Azure Government note: Query folding is marked **Gov-ready / Verify for source**. The concept is core Power Query, but hands-on folding validation requires a connector and source that support folding in the target environment.

Objective: Understand and validate folding behavior.

Tasks

1. Review which file-based transformations cannot fold.
2. If a SQL or folding-capable source is available, connect to a sample table.
3. Apply filters and column selection.
4. Use **View Native Query** where available.
5. Add a non-foldable step and observe the effect.

Expected result

Learners can explain why folding improves performance and why it must be validated per source.

Concept note: native queries and source systems

Native queries can push complex logic to a source system, but they also make the report dependent on source SQL, gateway configuration, identity, and security review. Prefer maintainable Power Query steps for the core lab. Use native queries only when the source, folding behavior, credentials, and Azure Government support have been validated.

Exercise 7: Incremental refresh preparation

Azure Government note: Incremental refresh preparation in Desktop is generally Gov-ready, but applying and running the policy in the Service is **Verify for Gov**.

Objective: Prepare a fact query for incremental refresh.

Tasks

1. Create DateTime parameters named `RangeStart` and `RangeEnd` .
2. Filter `FactOrders[OrderDate]` using `RangeStart` and `RangeEnd` .
3. Confirm the filtered column is a DateTime-compatible column.
4. Document the intended refresh and archive windows.
5. Validate Service requirements before making refresh policy setup mandatory.

Expected result

The fact query is ready for incremental refresh policy configuration when the Service environment supports it.

Validation checklist

- ☐ Raw and staging queries have load disabled.
- ☐ Final fact query has explicit data types.
- ☐ Folder combine filters to expected files only.
- ☐ Source file lineage is preserved.
- ☐ Custom text cleanup function handles null values.
- ☐ Data quality issues are identified and documented.
- ☐ Query folding is demonstrated or explained with source limitations.
- ☐ All five Lab 03 parameters are documented: `RawDataBaseUrl` , `SourceFolderPath` , `EnvironmentName` , `RangeStart` , and `RangeEnd` .
- ☐ Incremental refresh parameters are `DateTime` and correctly named.
- ☐ Dataflows, Dataflows Gen2, connectors, and Service refresh features include Gov notes.